2D 타워 디펜스 오픈소스 6개 분석

수업 템플릿 적합성 검토 · 원광대학교 게임콘텐츠학과

여섯 저장소를 전부 클론해 **스크립트 6,000여 줄을 직접 읽고** 작성했습니다. 씬 파일은 열지 않았으므로 "씬에만 있는 설정"은 확인 범위 밖입니다.

---

0. 결론 먼저

그대로 쓸 수 있는 것은 하나도 없습니다. 그리고 그 이유가 코드 품질이 아니라는 점이 중요합니다.

문제	해당
LICENSE 파일이 없다	6개 중 **4개**
웨이브를 데이터로 설계할 수 없다	**6개 전부**
ScriptableObject 사용 0건	**6개 전부**
에셋 출처 표기 0건	**6개 전부**
Unity 2017 ~ 2021	**6개 전부**

GitHub에서 라이선스가 없다는 것은 "마음대로 쓰라"가 아니라 "저작권 전면 보유"입니다. 수업 자료로 배포하거나 학생 과제의 기반으로 삼으려면 원저작자의 허락을 따로 받아야 합니다. 이 한 가지가 네 저장소를 선택지에서 지웁니다.

그리고 **여섯 개 모두 웨이브가 랜덤 생성입니다.** 타워 디펜스 수업에서 가장 가르치고 싶은 "3웨이브는 빠른 적 5마리, 5웨이브는 보스 1마리" 같은 **웨이브 테이블 설계**를 어느 저장소에서도 보여 줄 수 없습니다.

> **역설적으로, 지금 만드시는 한빛 14장 프로젝트가 이 여섯 개보다 낫습니다.** 그 프로젝트의 `EnemyWaveData`는 `[System.Serializable]`이 붙은 `List`라 **인스펙터에서 웨이브마다 적 수·생성 간격·대기 시간을 지정할 수 있습니다.** 여섯 개 중 어느 것도 못 하는 일입니다. 거기에 Unity 6.3이고, 코드를 직접 복원하며 만드셔서 모든 수치를 알고 계시고, 라이선스 걱정도 없습니다.

---

1. 한눈에 보기

저장소	Unity	규모	라이선스	가독성	구조	확장	수업
Rion5 / 2D-TowerDefense	2017.2	8파일 741줄	**없음**	3.5	2.5	3.0	2.5
JanWalsh91 / tower_defense	2017.3	23파일 1,272줄	**없음**	3.0	2.0	1.0	2.0
Evander / Tower-Defense	2018.3	8파일 748줄	MIT (코드만)	3.0	2.0	2.0	3.0
tomersh9 / 2D-Tower-Defense	2019.3	35파일 1,670줄	**없음**	3.5	3.0	2.0	2.5
caganoduncuoglu / StickyDefense	2021.2	10파일 607줄	**없음**	2.0	1.5	1.0	1.0
lawalm / Panda-Invasion	2018.3	19파일 910줄	MIT (표시 위반)	3.5	2.5	2.0	2.5

5점 만점. 자세한 근거는 아래 각 절에 있습니다.

---

2. 저장소별 상세

2-1. Rion5 / 2D-TowerDefense — 가장 작고 단순하지만 빌드가 안 된다

8파일 741줄. 여섯 개 중 가장 읽기 쉬운 분량입니다.

	내용
적 이동	웨이포인트 7개 + **체크포인트 트리거 충돌**로 인덱스 전진
타워 배치	버튼 선택 → 맵 클릭. 타일 **태그를** `buildSite` → `buildSiteFull` **로 바꿔** 중복 방지
있는 것	웨이브(단순), 골드, UI, 사운드(클립만)
없는 것	업그레이드, 판매, 저장, 레벨 전환

쓸 만한 부분 — `[SerializeField] private` + public 게터 패턴이 전 파일에 일관되고, 레이캐스트·싱글턴 개념을 "왜"까지 설명하는 영어 주석이 있습니다. `Projectile.cs`·`TowerButton.cs`는 **캡슐화 수업 예제로 그대로 쓸 수 있습니다.**

지뢰

- `Assets/Pluggins/VSCode.cs`(1342줄)가 **Editor 폴더 밖에서** `using UnityEditor;`**를 가드 없이** 씁니다. 에디터에서는 돌지만 **빌드하면 컴파일 에러**입니다. 폴더명도 "Pluggins" 오타라 Unity 특수 폴더 취급도 못 받습니다
- **빌드 씬 목록이 비어 있습니다.** 빌드하면 씬이 하나도 안 들어갑니다
- `Singleton<T>`의 else 분기가 `Destroy(FindObjectOfType<T>())` — **매니저를 스스로 파괴**합니다. "왜 매니저가 사라지나" 디버깅 지옥의 씨앗
- `Tower.cs`의 `Destroy(newProjectile)`은 **컴포넌트만** 파괴합니다. 스프라이트가 씬에 영구 잔존
- 적이 죽어도 리스트에서 빠지지 않아 **시체가 쌓입니다.** 타워가 매 프레임 그걸 순회합니다

- 적 프리팹의 웨이포인트가 **씬이 아니라 프리팹 에셋을 참조**합니다. 씬에서 경로를 옮겨도 적은 안 바뀝니다 → **레벨 디자인 과제에 부적합**
- 저자 본인이 README에 "Work in Progress", "Need to fix enemy spawning bug"라고 적어 두었습니다

한 줄 평 — 분량은 이상적이지만, 교사가 1~2일 정비한 뒤 **버그 수정 실습 교재**로 쓸 베이스입니다.

2-2. JanWalsh91 / tower_defense — UI 기법은 볼 만하나 협업이 불가능하다

23파일 1,272줄. 프랑스 Ecole 42의 과제물입니다. 저장소에 **42 과제 명세서 PDF가 그대로 커밋**되어 있어, 프랑스어 주석이 달린 파일들이 학교가 제공한 스켈레톤 코드임을 알 수 있습니다.

	내용
적 이동	체크포인트가 다음 체크포인트를 가리키는 **단방향 연결 리스트**
타워 배치	**드래그 앤 드롭** + 키보드(Q/W/E)+클릭 **두 경로**
있는 것	업그레이드(프리팹 교체), 판매(구매가/2 환불), **방사형 메뉴**, 게임 속도 조절, 랭크(F~S+)
없는 것	사운드 전무, 저장 전무

쓸 만한 부분

- **방사형 메뉴**(`towerRadialMenu.cs`) — 우클릭으로 타워 주위에 업그레이드/판매 버튼을 띄웁니다
- **에디터 확장 예제 두 개** — 1단위 그리드 자동 스냅 윈도우(`Assets/Editor/AutoSnap.cs`), 기즈모 경로 시각화(`visualPath*.cs`). **"에디터 확장" 차시 교재로 바로 쓸 수 있습니다**
- `OnEnable` / `OnDisable`에서 이벤트 구독·해제를 **짝 맞춰** 처리한 드문 예

지뢰

- **씬과 프리팹이 전부 바이너리 직렬화 상태입니다.** git diff·merge가 불가능합니다. **학생 팀 프로젝트에 치명적**이고, 에디터를 열지 않으면 타워 수치를 볼 수조차 없습니다
- **폭탄으로 마지막 적을 죽이면 승리 판정이 영원히 안 납니다.** 폭탄이 `Destroy`로 적을 직접 지워서 사망 처리 함수를 우회합니다
- 로켓 비행 중 다른 타워가 그 적을 먼저 죽이면 **NullReference**
- `transform.GetChild(3)`로 UI 패널을 찾습니다. 캔버스 자식 순서를 바꾸면 조용히 깨집니다
- 폰트가 `Bad Unicorn DEMO.ttf` — "DEMO"는 보통 비상업 평가판이고 재배포 금지입니다
- `download.jpg`, `unicorn.jpg` 같은 **웹에서 그대로 받은 티가 나는 이미지**가 섞여 있습니다
- 타워 1종 추가에 **코드 5개 파일 수정 + 인스펙터 수동 연결 15곳 이상 × 씬 4개 반복**이 필요합니다

한 줄 평 — 부분 발췌해 기법 참고용으로는 좋지만, 템플릿으로는 바이너리 씬 하나만으로도 탈락입니다.

2-3. Evander / Tower-Defense — 법적으로 가장 안전하지만 주석이 인도네시아어

8파일 748줄, MIT. 여섯 개 중 **코드 라이선스가 가장 깨끗합니다.**

	내용
적 이동	웨이포인트 `Transform[]`, 전진 판정이 **적이 아니라 LevelManager에** 있음
타워 배치	미리 깔아둔 슬롯 + **UI 드래그 앤 드롭**
있는 것	오브젝트 풀링(적·총알), 스플래시 데미지, 경로 기즈모
없는 것	**웨이브 개념 자체가 없음, 골드 없음**(타워가 무료·무제한), 업그레이드, 판매, 저장

쓸 만한 부분

- 네이밍이 748줄 내내 흔들림 없습니다(`_camelCase` 필드, `PascalCase` 메서드, `{ get; private set; }`)
- **오브젝트 풀링이 실제로 구현**되어 있어 좋은 수업 소재입니다
- 주석이 "무엇"이 아니라 "왜"를 설명합니다 — 도착 판정에 왜 0.1을 쓰는지, 왜 `FixedUpdate`인지

지뢰

- **주석이 100% 인도네시아어입니다.** 위에 장점으로 적은 주석이 한국 학생에게는 전부 무용지물입니다. **전량 번역이 선행 작업**
- `LevelManager.cs` 243줄이 **God 클래스**입니다. 스폰 + 풀링 + 경로 + 타워 루프 + 적 루프 + 데미지 + HUD + 승패를 한 파일이 쥐고 있고, `Update()` 하나가 **59줄**입니다
- 반대로 `Enemy`와 `Tower`는 스스로 `Update`를 돌리지 않는 **수동적 컴포넌트**입니다. 학생이 "적 동작은 Enemy.cs에 있겠지" 하고 열면 비어 있습니다
- 총알 풀이 `name.Contains(prefab.name)` **문자열 부분일치**로 재사용 대상을 찾습니다. `Bullet`을 요청하면 `Bullet Variant 2`가 걸립니다 — **엉뚱한 총알이 날아갑니다**
- README가 **"All the code available under the MIT licenses"** 라고 **코드로 한정**합니다. 스프라이트 15개·오디오 2개의 출처는 어디에도 없습니다

한 줄 평 — 코드만 쓰면 법적으로 안전하고, **"God 클래스 리팩터링" 과제의 출발점으로는 여섯 개 중 가장 적합합니다.**

2-4. tomersh9 / 2D-Tower-Defense — 구조는 가장 좋으나 에셋이 위험하다

35파일 1,670줄. 폴더가 `Managers` / `Enemy Related` / `Tower Related` / `Wave Spawning` / `Nodes` / `Shop` / `UI` / `Settings`로 나뉘어 있고 **네임스페이스까지 씁니다.** 입문 프로젝트에서는 드문 수준입니다.

	내용
적 이동	웨이포인트. `Path` 프리팹의 자식 Transform을 순서대로 수집
타워 배치	상점 클릭 → **고스트가 마우스를 따라다님** → 노드 클릭
있는 것	판매(비용/1.5 환불), 레벨 4개 + 해금, 설정 저장(PlayerPrefs), BGM·SFX, 배속
없는 것	**업그레이드 전무**("upgrade" 문자열 0건), 게임 진행 세이브

쓸 만한 부분

- **컴포넌트 분해가 교과서적입니다.** 타워 1기가 `Tower`(비용) + `Weapon`(발사) + `EnemyScanner`(탐색) + `DamageDealer`(피해)로 쪼개져 있고, 특히 `DamageDealer.cs`는 **8줄**짜리인데 타워 투사체와 적 양쪽에서 재사용됩니다. **"단일 책임 컴포넌트" 설명 자료로 그대로 쓸 수 있습니다**
- 이벤트 구독·해제를 `OnEnable` / `OnDisable` 짝으로 올바르게 처리합니다
- 파일 평균 48줄, 최대 202줄. 거대 클래스가 없습니다

지뢰

- **싱글턴이 5가지 방식으로 난립합니다.** 순수 C# lazy / `FindObjectsOfType` 중복검사 + `DontDestroyOnLoad` / `GetInstance()` 없는 것 / `MonoBehaviour`인데 전부 static인 것. **"싱글턴"이라는 말 하나로 다섯 가지를 가르쳐야 합니다**
- `Wave` / `SubWave`**가** `[Serializable]`**조차 아닌 순수 C# 클래스입니다.** 인스펙터에 노출이 **구조적으로 불가능**해서, 웨이브 구성을 지정할 통로가 코드에 없습니다. 난이도 임계값(10웨이브·30웨이브)도 하드코딩
- **11웨이브에서 적이 15마리 → 5마리로 급감합니다.** Medium/Hard 구간에 `waveIndex`를 더하는 것을 빠뜨린 실수
- `Weapon`의 발사가 **컴포넌트 실행 순서에 의존**합니다. Unity가 보장하지 않는 순서라 **우연히 동작하는 코드**이고, 학생이 컴포넌트를 재부착하면 타워가 조용히 발사를 멈춥니다
- **Level 4를 클리어하면** `buildIndex + 1`**이 범위를 넘어 예외**가 납니다
- **에셋이 가장 위험합니다.** `Fantasy Wooden GUI Free`(Asset Store, **PSD 원본까지 포함**), `0x72_DungeonTilesetII`(itch.io), `Serene_Village_16x16`**(itch.io, 통상 유료)**, `pipo-btleffect`(일본 ぴぽや 소재) — 출처 표기가 **하나도 없습니다**
- `Assets/TextMesh Pro/Examples & Extras/`에 TMP 예제 **33개**가 남아 있어, 학생이 보는 .cs 파일은 35개가 아니라 **68개**입니다

한 줄 평 — 코드 구조는 인용할 가치가 있지만, 유료 의심 에셋 때문에 **저장소를 그대로 배포하면 저작권 리스크가 학교에 전가됩니다.**

2-5. caganoduncuoglu / StickyDefense — 간판만 있고 알맹이가 없다

10파일 607줄. `FloatSO` · `GameEvents` · `SaveSystem` 같은 파일명만 보면 설계가 좋아 보이지만, 열어보면 다릅니다.

	내용
적 이동	**길찾기가 없습니다.** DOTween 콜백 **5중 중첩**으로 절대좌표를 하드코딩
타워 배치	**플레이어가 위치를 고르지 않습니다.** 버튼을 누르면 빈 슬롯 중 **무작위**로 배치
없는 것	업그레이드, 판매, 사운드 전무, 게임오버/승리 화면, 패배 판정

지뢰

- `SpawnTower()`의 `else { SpawnTower(); }` — **무한 재귀입니다.** 코인이 모자라거나 8칸이 다 찬 상태에서 버튼을 누르면 **StackOverflow로 에디터가 그대로 죽습니다.** 첫 수업에서 거의 확실히 발생합니다
- `UIManager`가 **필드 초기화자에서** `GameObject.Find`**를 호출**합니다. Unity가 금지하는 패턴이라 씬 진입 즉시 예외가 납니다
- 적 경로가 **절대좌표 하드코딩**이라 "적이 다른 길로 가게 해 보자"는 가장 자연스러운 첫 과제가 **리팩터링 과제가 되어 버립니다**
- 저장이 `BinaryFormatter` 입니다. Microsoft가 보안 취약으로 사용 중단을 권고한 API로, **수업에서 가르치면 안 되는 것**입니다. 경로도 `Application.persistentDataPath`가 아닌 상대경로라 플랫폼에 따라 쓰기 실패
- `ScriptableObject`는 쓰지만 **씬 간 값 전달 하나**에만 씁니다. 타워 스탯은 `Awake()`에서 하드코딩으로 덮어써서 **인스펙터 값이 무시됩니다**
- 저장소에 **남의 세이브 파일**(`Saves/saveData.bin`)이 커밋돼 있습니다
- 외부 라이브러리 **DOTween이 번들**되어 있습니다(자체 라이선스)

한 줄 평 — 실습 템플릿으로 부적합합니다. 버그가 작고 명확해서 **"버그 리포트 쓰기" 과제 소재**로만 쓸모가 있습니다.

2-6. lawalm / Panda-Invasion — 책 예제인데, 원본을 쓰는 편이 낫다

19파일 910줄, MIT. README가 밝히듯 **Packt 출판사의 책『Getting Started with Unity 5.x 2D Game Development』예제**를 거의 완성한 것입니다.

	내용
적 이동	웨이포인트 단방향 연결 리스트
타워 배치	마우스에 붙어 다니다 클릭하면 고정. 그리드·스냅 없음
있는 것	**구매·판매·업그레이드를 추상 클래스 상속으로 분리**
없는 것	사운드, 저장, 레벨 전환, **패배 조건**

쓸 만한 부분

- `TradeCupcakeTower` 추상 클래스 + 3개 상속(구매/판매/업그레이드)은 **상속과 다형성 수업 예제로 그 자체로 훌륭합니다**
- `[Header]`, `[Range]`, `[RequireComponent]`, `Animator.StringToHash` 캐싱, `OnDrawGizmos` 사거리 표시 — 입문서가 가르치는 좋은 습관이 남아 있습니다
- 주석이 전체의 17%이고 "왜"를 설명합니다

지뢰 — 버그가 특히 많고, 전부 명확합니다

- **업그레이드/판매 버튼이 첫 프레임에 사라져 영원히 안 돌아옵니다.** `Update()`에서 자기를 `SetActive(false)` 하는데, **비활성 오브젝트는** `Update`**가 안 돌아 스스로를 다시 켤 수 없습니다**
- 판매하면 설탕은 받는데 **타워가 화면에 남습니다.** `Destroy(컴포넌트)`를 했기 때문

- 업그레이드 조건문이 **반전**되어 있어 1회만 업그레이드됩니다
- 웨이브 코루틴이 **1웨이브 후 영구 정지**합니다. 기다리는 변수를 감소시키는 코드가 없습니다
- **플레이어가 절대 지지 않습니다.** 적이 끝점에 닿아도 체력이 안 깎입니다
- **스포너가 이중화**되어 있어 둘 다 씬에 있으면 적이 2배로 나오고 곧바로 승리 화면이 뜹니다
- 19개 중 **5개가 빈 껍데기 또는 중복**입니다. 학생이 `MoveEnemy.cs`를 고치고 "왜 안 바뀌죠?"라고 묻게 됩니다
- **빌드 씬 목록이 존재하지 않는 `SampleScene.unity` 하나**를 가리킵니다. 빌드하면 빈 씬만 나옵니다
- **MIT인데 Packt의 원 저작권 표시가 삭제**되고 저장소 주인 이름으로 바뀌어 있습니다. MIT 조건 위반입니다

> **중요** — Packt의 공식 예제 저장소도 MIT이고 **저작권 체인이 깨끗합니다.** `github.com/PacktPublishing/Getting-Started-with-Unity-5.x-2D-Game-Development` **이 포크 대신 공식 저장소를 쓰시면 됩니다.**

한 줄 평 — 그대로는 "안 되는 템플릿"이지만, **버그 12건이 전부 명확해서 "버그 찾기 과제"로는 특급 교재**입니다.

---

3. 여섯 개를 가로질러 본 것

3-1. 라이선스 — 가장 큰 벽

네 저장소에 LICENSE가 없습니다. MIT가 있는 둘도 조건이 붙습니다.

저장소	상태	수업 사용
Rion5, JanWalsh91, tomersh9, caganoduncuoglu	**없음**	**원저작자 허락 필요**
Evander	MIT, 단 README가 "코드만"이라고 한정	코드는 가능, **에셋은 교체**
lawalm	MIT, 단 원 저작권 표시 삭제	**Packt 공식 저장소를 쓸 것**

에셋 출처 표기는 여섯 개 모두 0건입니다. tomersh9에는 유료로 보이는 itch.io 타일셋까지 섞여 있습니다.

3-2. 웨이브 — 여섯 개 모두 랜덤이다

저장소	웨이브 생성 방식
Rion5	`totalEnemies += waveNumber`, 적 종류는 `Random.Range(0, waveNumber)`
JanWalsh91	웨이브마다 적 **1종을 랜덤 선택**, HP는 선형 증가 공식
Evander	**웨이브 개념 자체가 없음.** 5초마다 15마리를 흘려보냄
tomersh9	난이도 그룹에서 **무작위 1종**, 서브웨이브는 무조건 2개
caganoduncuoglu	웨이브 수·크기만 있음
lawalm	`numOfPandsPerWave += 3` 한 줄

"이번 웨이브는 이렇게 구성한다"를 설계자가 지정할 수 있는 저장소가 하나도 없습니다. 플레이할 때마다 구성이 달라집니다.

3-3. 타워 추가에 코드가 필요 없다 — 수업 관점의 역설

Evander, tomersh9, lawalm 세 곳은 **타워 1종 추가가 프리팹 복제 + 인스펙터 작업만**으로 끝납니다. 보통은 장점입니다.

그런데 게임프로그래밍 수업 과제로는 약점입니다. 학생이 타워를 추가해도 C# 코드를 한 줄도 쓰지 않기 때문에 **프로그래밍 학습이 일어나지 않습니다.** 반대로 JanWalsh91은 코드 5개 파일을 고쳐야 해서 과제로는 너무 무겁습니다.

3-4. 공통된 코드 냄새

여섯 개에서 반복해서 나온 것들입니다. **그대로 "하면 안 되는 예" 목록이 됩니다.**

	몇 곳에서	내용
`FindObjectOfType`	6/6	Unity 2023.1부터 deprecated
싱글턴 오남용	6/6	중복 방어·해제 없음. tomersh9는 **5가지 방식 혼재**
문자열 태그·이름 의존	6/6	`GameObject.Find("/Canvas/...")`, `tag == "..."`, `name.Contains(...)`
매직 넘버	6/6	**상수 선언이 하나도 없는** 저장소가 둘
`Destroy(컴포넌트)` 혼동	3곳	GameObject가 아니라 컴포넌트를 지워 스프라이트가 남음
빈 `Start()` / `Update()`	6/6	Unity가 매 프레임 호출하는 빈 함수

---

4. 수업에서 쓰는 네 가지 방법

방법 A · 그대로 템플릿으로 — 불가능

라이선스, Unity 버전, 버그 세 가지가 동시에 막습니다. 권하지 않습니다.

방법 B · 코드만 발췌해 참고 자료로 — 권장

주제	가져올 곳
단일 책임 컴포넌트	tomersh9의 `DamageDealer.cs`(8줄), `Tower` / `Weapon` / `EnemyScanner` 분해
상속과 다형성	lawalm의 `TradeCupcakeTower` 추상 클래스 + 3개 상속
오브젝트 풀링	Evander의 `GetBulletFromPool()`
캡슐화	Rion5의 `[SerializeField] private` + 게터 패턴
에디터 확장	JanWalsh91의 `AutoSnap.cs`(EditorWindow), `visualPath*.cs`(기즈모)
드래그 앤 드롭 UI	Evander의 `TowerUI.cs`, JanWalsh91의 `UIDragTower.cs`
방사형 메뉴	JanWalsh91의 `towerRadialMenu.cs`

코드 조각을 강의 슬라이드에 인용하는 것은 저작권법상 인용에 해당할 여지가 크지만, 저장소 전체를 복사해 배포하는 것은 다릅니다. 출처를 반드시 밝히시고, 분량은 설명에 필요한 만큼만 쓰시는 편이 안전합니다.

방법 C · 버그 찾기 / 리팩터링 과제 — 가장 추천

이 여섯 개의 **진짜 가치는 여기에 있습니다.** 버그가 실제로 있고, 원인이 명확하고, 고치면 눈에 보이게 달라집니다.

저장소	과제
lawalm / Panda-Invasion	**버그 12건 찾기.** "업그레이드 버튼이 왜 사라지나", "판 타워가 왜 화면에 남나", "왜 지지 않나" — 전부 증상이 눈에 보이고 원인이 한 줄입니다
Evander / Tower-Defense	**God 클래스 리팩터링.** 243줄 `LevelManager`를 Spawner / Pool / HUD로 쪼개기. MIT라 법적으로도 안전
tomersh9 / 2D-Tower-Defense	**ScriptableObject 전환.** `Wave` / `SubWave`를 `[CreateAssetMenu]` 에셋으로 바꾸기. 변경 지점이 4개 파일로 국소적이라 과제 크기가 적절
caganoduncuoglu / StickyDefense	**버그 리포트 작성.** 재현 절차·기대 동작·실제 동작을 양식에 맞춰 쓰는 연습

방법 D · 설계 비교 분석 과제 — 흥미롭습니다

같은 게임을 여섯 팀이 어떻게 다르게 풀었는가를 비교하게 하는 과제입니다.

- 타워를 어디에 세우게 했는가 — 태그 전환 / 슬롯 오브젝트 / 노드 / 랜덤 / 자유 배치

- 적을 어떻게 움직였는가 — 웨이포인트 배열 / 연결 리스트 / 트리거 충돌 / DOTween 하드코딩
- 누가 적을 움직이는가 — 적 자신 / 매니저가 대신
- 웨이브를 어떻게 만들었는가 — 전부 랜덤이다. **왜 아무도 테이블을 만들지 않았을까?**

마지막 질문이 특히 좋습니다. "랜덤이 쉽기 때문"이라는 답에 도달하면, **쉬운 선택이 어떻게 게임의 설계 여지를 없애는지**를 스스로 발견합니다.

---

5. 추천

지금 만드시는 한빛 14장 프로젝트를 템플릿으로 쓰시고, 이 여섯 개는 과제 재료로 쓰시기를 권합니다.

이유는 세 가지입니다.

첫째, 웨이브를 데이터로 설계할 수 있는 유일한 것입니다. `EnemyWaveData`가 `[System.Serializable]` 리스트라 인스펙터에서 웨이브마다 지정할 수 있습니다. 여섯 개 중 어느 것도 못 합니다.

둘째, 법적으로 깨끗합니다. 교재 기반 수업용 자료이고, 에셋도 출처가 분명합니다.

셋째, 교수님이 모든 줄을 알고 계십니다. 빠진 코드를 복원하고 버그를 고치며 만드셨고, GDD와 밸런싱 수치까지 문서로 있습니다. 남의 저장소를 쓰면 "이 수치가 왜 이렇죠?"에 답할 수 없는 순간이 반드시 옵니다.

그리고 이 여섯 개는 이렇게 쓰시면 됩니다.

용도	저장소
버그 찾기 과제	**lawalm / Panda-Invasion**
리팩터링 과제	**Evander / Tower-Defense** (MIT)
ScriptableObject 전환 과제	**tomersh9 / 2D-Tower-Defense**
구조 설계 참고	**tomersh9** (컴포넌트 분해), **lawalm** (상속)
에디터 확장 참고	**JanWalsh91**
비교 분석 과제	**여섯 개 전부**

---

6. 앞으로 다른 저장소를 볼 때의 체크리스트

이번 분석에서 실제로 판가름을 낸 항목만 추렸습니다. **이 다섯 개를 5분 안에 확인하면 대부분 걸러집니다.**

#	확인할 것	어디서	아니면
1	**LICENSE 파일이 있는가**	저장소 루트	없으면 배포 불가. 여기서 끝
2	**에셋 출처가 표기돼 있는가**	README, CREDITS	없으면 에셋 전부 교체 각오
3	**씬·프리팹이 텍스트인가**	`.unity` 파일 첫 줄이 `%YAML`인가	바이너리면 git merge 불가 → 팀 과제 불가
4	**웨이브가 데이터인가**	웨이브 클래스에 `[Serializable]`이 있는가	없으면 웨이브 설계를 못 가르침
5	**Unity 버전 차이가 몇 년인가**	`ProjectSettings/ProjectVersion.txt`	5년 넘으면 업그레이드가 수업 전 작업이 됨

세 번째는 터미널에서 한 줄로 확인됩니다.

```
head -1 Assets/Scenes/*.unity
```

`%YAML 1.1`로 시작하면 텍스트, 알 수 없는 바이트가 나오면 바이너리입니다.